



Aula 1 – Nivelamento



Algoritmos e Estruturas de Dados

Dominando C++, Estruturas e Lógicas para ICPC e Entrevistas

Objetivo: Igualar o conhecimento base para as aulas seguintes.



Cronograma

- | | |
|--------------------------------------|--------|
| 1. Sintaxe & Lógica Essencial | 8 min |
| 2. Estruturas de Dados | 20 min |
| 3. Extras (String/Ordenação Custom.) | 5 min |
| 4. Matemática Útil | 5 min |
| 5. Interpretação & Simulação | 10 min |
| 6. Encerramento & Direcionamento | 5 min |

Já domina o conteúdo?

Aproveite para prestar atenção nas dicas de boas práticas e ganchos para as aulas avançadas.

Viu algo novo?

Anote os conceitos e consulte o material de apoio



Tipos Primitivos e Overflow



CÓDIGO INCORRETO (OVERFLOW)

```
int a = 1000000;  
int b = 1000000;  
long long ans = a * b; // bug!  
// ans recebe valor negativo.  
// (-727379968)
```



CÓDIGO CORRETO

```
long long a = 1000000;  
long long b = 1000000;  
long long ans = a * b; // ok!  
// ans = 1000000000000 (10^12)
```



Input & Output

```
// Template Rápido de E/S em C++
#include <bits/stdc++.h>
using namespace std;

int main() {
    // Otimização I/O essencial
    ios_base::sync_with_stdio(false); cin.tie(0);
    int t;

    if (cin >> t) {
        while (t--) {
            // Resolver caso de teste
        }
    }
    return 0;
}
```




Recursão Básica

```
// Exemplo de Recursão Simples
int soma(int n) {
    // 1. Caso Base (Parada)
    if (n <= 0) return 0;

    // 2. Passo Recursivo
    return n + soma(n - 1);
}
```

Caso Base:

Determina quando parar. Sem ele, ocorre Stack Overflow.

Passo Recursivo

Reduz o problema para uma instância menor.



Vector & Ordenação

OPERAÇÕES BÁSICAS

```
vector<int> v;  
v.push_back(10); // O(1)  
int val = v[0];  // O(1)  
int sz = v.size();
```

ORDENAÇÃO PADRÃO

```
// Crescente  $O(N \log N)$   
sort(v.begin(), v.end());
```

INVERSÃO DE ELEMENTOS

```
// Inverte ordem dos elementos  
reverse(v.begin(), v.end());
```

COMPARADOR CUSTOMIZADO

```
// Decrescente rápido  
sort(v.begin(), v.end(),  
    greater<int>());
```



Map e Set (Ordered vs. Unordered)

Estrutura	Use quando...	Não use se...
set map	Precisa manter ordem, buscando mínimo/máximo, ou percorrer em ordem.	Ordem não importa – desperdiça velocidade.
unordered_set unordered_map	Precisa checar presença ou contar, rápido, sem se importar com ordem.	Precisa de mín./máx. ou percorrer ordenado.



Map e Set (Ordered vs. Unordered)

```
#include <bits/stdc++.h>

unordered_map<int, int> freq;
for (int i = 0; i < n; i++) {
    int x;
    cin >> x;
    freq[x]++; // O(1) em média
}

// Iterar pelo map
for (auto [val, conta] : freq) {
    cout << val << ": " << conta << "\n";
}
```

PROBLEMA:

"Dada uma sequência de N inteiros, calcule quantas vezes cada valor aparece na entrada."

input

4 2 4 5 2 4

output

2: 2

4: 3

5: 1



Stack & Queue

stack (LIFO)

Último a entrar, primeiro a sair (Pilha de pratos).

```
stack<int> st;  
st.push(10);  
int topo = st.top(); // 10  
st.pop();
```

Uso futuro: DFS.

queue (FIFO)

Primeiro a entrar, primeiro a sair (Fila de banco).

```
queue<int> q;  
q.push(10);  
int f = q.front(); // 10  
q.pop();
```

Uso futuro: BFS em Grafos.



Menção Rápida: Pair, Tuple e Deque

pair<A, B>

Guarda 2 valores juntos (ex: x, y)

```
pair<int,int> p = {1, 2};  
int x = p.first;  
cout << x;
```

Saída: 1

deque

Fila com $O(1)$ nas duas pontas.

```
deque<int> dq = {2, 3};  
dq.push_front(1);  
dq.push_back(4);
```

Agora dq é {1, 2, 3, 4}

tuple<A, B, C>

Guarda 3 ou mais valores

```
tuple<int, int, int> t(1,2,3);  
cout << get<1>(t); // get<N>(t)  
// retorna o N elemento de t.
```

Saída: 2



Manipulação de Strings

Operações Comuns

```
string s = "Algoritmos";  
int tam = s.length();  
string sub = s.substr(0, 4);  
cout << sub;
```

Saída: "Algo"

⚠ Alerta de Cópia $O(N^2)$

Evite `s = s + c` em loops! Use `s += c` ou `s.push_back(c)` para evitar recriar a string a cada iteração.



Ordenação Customizada

```
// Ordenar por idade decrescente, desempate por nome crescente
vector<pair<int, string>> pessoas = {{25, "Ana"}, {30, "Bruno"}, {25, "Carlos"}};

sort(pessoas.begin(), pessoas.end(), [](pair<int,string> a, pair<int,string> b) {
    if (a.first != b.first) {
        return a.first > b.first; // maior idade primeiro
    }
    return a.second < b.second; // desempate por nome
}); // Agora "pessoas" é: {30, "Bruno"}, {25, "Ana"}, {25, "Carlos"}.
```




Teoria dos Números

1. Divisibilidade
2. Números Primos/Crivo
3. Algoritmo de Euclides (MDC/MMC)
4. Aritmética Modular
5. Exponenciação Binária



1

Ler as Restrições Primeiro

O tamanho de N limita os algoritmos possíveis.

2

Identificar Edge Cases

Testar $N=1$, valores zerados, negativos, limites extremos.

3

Simular no Rascunho

Rode os exemplos do enunciado na mão antes de codar.



Estimando Complexidade

Valor de N	Complexidade Esperada	Exemplo de Técnica
$N \leq 10 \sim 12$	$O(N!)$ ou $O(2^N)$	Backtracking/Força Bruta
$N \leq 500 \sim 10^3$	$O(N^2)$	Loops Aninhados / DP Simples
$N \leq 10^5 \sim 10^6$	$O(N \log N)$ ou $O(N)$	Ordenação / Prefix Sum / Map
$N \geq 10^8$	$O(\log N)$ ou $O(1)$	Busca Binária / Matemática



PROVOCAÇÃO CLÁSSICA

“A cada segundo, o jogador A anda X passos. Se encontrar B, inverta a direção...”

Qual algoritmo devemos usar aqui?

Nenhum! Apenas programe o processo passo a passo com o uso correto das estruturas de dados!



Material de Apoio em C++

Resumos de sintaxe, STL Cheat Sheet e gabaritos de códigos.

github.com/pedrodev-bot/ufabc-revisao-icpc-2026



Lista Recomendada

Exercícios selecionados por tag no Codeforces/Leetcode.

[/exercicios/Aula 1.md](#)



Encerramento

Algoritmos e Estruturas de Dados



Canal de Dúvidas aberto no Discord/WhatsApp.



Próxima Aula terá data definida pelo WhatsApp.